1. Конфигурация сети, средства устранения неполадок и отладки

1.1 ifconfig

ifconfig - это инструмент для настройки сетевого интерфейса, который также используется и для инициализации интерфейсов во время загрузки системы. После того, как сервер запущен и работает, его можно использовать для назначения IP-адреса интерфейсу и включения или отключения интерфейса по запросу.

Также ifconfog используется для просмотра статуса IP-адреса, оборудования/MAC-адреса, размера MTU активных в данный момент интерфейсов.

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 00:00:2d:3a:2a:28  txqueuelen 1000  (Ethernet)
        RX packets 5624  bytes 2226127 (2.1 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1691  bytes 176359 (172.2 KiB)
        TX errors 0  dropped 2 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 43  bytes 4078 (3.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 43  bytes 4078 (3.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Примеры применения:

```
┌──(kali㉿kali)-[~]
└─$ ifconfig -a
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        ether 00:00:2d:3a:2a:28  txqueuelen 1000  (Ethernet)
        RX packets 5624  bytes 2226127 (2.1 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1691  bytes 176359 (172.2 KiB)
        TX errors 0  dropped 2 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 43  bytes 4078 (3.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 43  bytes 4078 (3.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(root㉿kali)-[~]
└─# ifconfig eth0 192.168.56.5 netmask 255.255.255.0

┌──(root㉿kali)-[~]
└─# ifconfig eth0 up

┌──(root㉿kali)-[~]
└─# ifconfig eth0 down

┌──(root㉿kali)-[~]
└─#
```

Технически ifconfig устарел, и ему на замену пришел ip.

1.2 ping

ping (Packet INternet Groper) - это утилита для проверки возможности соединения между двумя системами. Она использует ICMP (протокол управляющих сообщений Интернета) для связи между узлами.

Мы можем при помощи этой утилиты проверить возможность подключения к ресурсу, например:

```
┌──(root㉿kali)-[~]
└─# ping -c 5 ya.ru
PING ya.ru (5.255.255.242) 56(84) bytes of data.
64 bytes from ya.ru (5.255.255.242): icmp_seq=1 ttl=243 time=56.6 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=2 ttl=243 time=56.1 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=3 ttl=243 time=55.8 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=4 ttl=243 time=56.1 ms
64 bytes from ya.ru (5.255.255.242): icmp_seq=5 ttl=243 time=63.8 ms

--- ya.ru ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4027ms
rtt min/avg/max/mdev = 55.802/57.667/63.793/3.072 ms
```

Как мы видим, доступ к ресурсу у нас есть.

1.3 traceroute

traceroute - это утилиты для отслеживания полного пути от локальной системы к другой сетевой системе. Он печатает количество переходов на этом пути, по которому мы перемещаемся, чтобы разобраться до конечного сервера. Зачастую эта утилита применяется после ping.

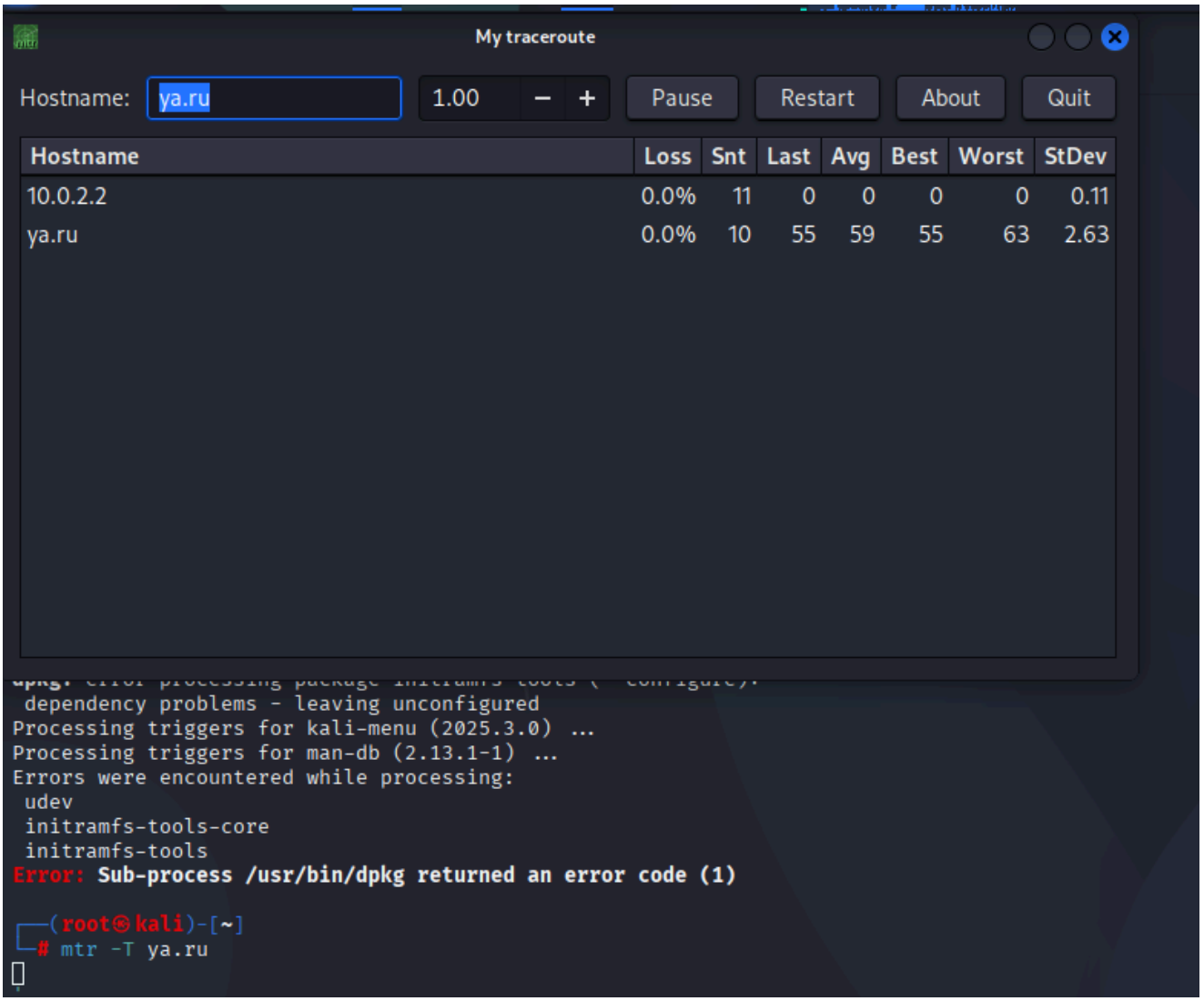
Попробуем отследить пакеты от локальной системы к серверу ya.ru.

```
┌──(root㉿kali)-[~]
└─# traceroute -T ya.ru
traceroute to ya.ru (5.255.255.242), 30 hops max, 60 byte packets
 1  10.0.2.2 (10.0.2.2)  0.374 ms  0.293 ms *
 2  ya.ru (5.255.255.242)  59.290 ms  59.662 ms  61.115 ms
```

1.4 MTR

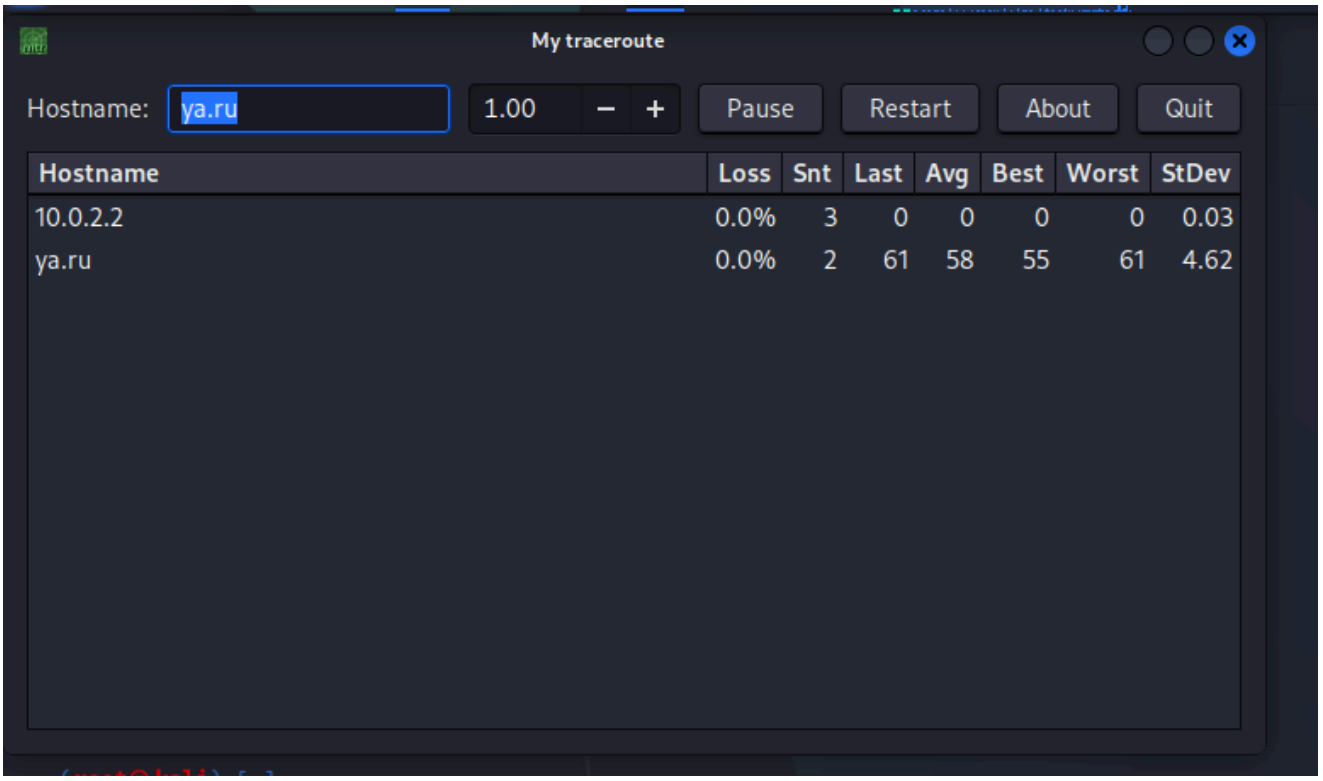
MTR - это современный инструмент диагностики сети, который сочетает в себе функции ping и traceroute. Его вывод обновляется в реальном времени по умолчанию, пока мы не выйдем из программы, нажав q.

Простой способ запустить MTR - это предоставить ему в качестве аргумента имя хоста или IP-адрес.
После запуска появится новое окно с выводом в реальном времени.



В нашем случае для закрытия не надо нажимать q - достаточно просто закрыть окно программы.

Можно также ограничить количество проверок и автоматически выйти из программы после окончания проверок, например:




```
apt install python3-q-text-as-dat

┌──(root㉿kali)-[~]
└─# mtr -T -c 5 ya.ru
```

1.5 route

route - это инструмент для отображения или управления таблицей IP-маршрутизации системы Linux. Он используется для настройки статических маршрутов к определенным хостам или сетям через интерфейс.

Набрав route, можно посмотреть таблицу IP-маршрутизации ядра.

```
┌──(root㉿kali)-[~]
└─# route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
default         10.0.2.2        0.0.0.0         UG    100    0        0 eth0
10.0.2.0        0.0.0.0         255.255.255.0   U     100    0        0 eth0

┌──(root㉿kali)-[~]
```

Добавим шлюз по умолчанию в таблицу.

```
┌──(root㉿kali)-[~]
└─# route add default gw 10.0.2.0
```

Добавим сетевой маршрут в таблицу маршрутизации, а затем удалим.

```
┌──(root㉿kali)-[~]
└─# sudo route add -net 192.168.1.0 netmask 255.255.255.0 eth0

┌──(root㉿kali)-[~]
└─# sudo route del -net 192.168.1.0 netmask 255.255.255.0 eth0

┌──(root㉿kali)-[~]
└─#
```

Также можно использовать ip route для запрета сетевого трафика на конкретном хосте.

```
┌──(root㉿kali)-[~]
└─# route add -host 10.10.2.20 reject

┌──(root㉿kali)-[~]
```

Чтобы отклонить сетевые пакеты для всей сети можно использовать команду route add -net 192.168.127.0 netmask 255.255.255.0 reject.

1.6 Nmcli

Nmcli - простой в использовании инструмент командной строки с поддержкой сценариев для сообщения информации о состоянии сети, управления сетевыми подключениями и управления NetworkManager.

Введя nmcli dev status можно увидеть все свои сетевые устройства.

```
┌──(root㉿kali)-[~]
└─# nmcli dev status
DEVICE  TYPE      STATE                   CONNECTION
eth0    ethernet  connected               Wired connection 1
lo      loopback  connected (externally)  lo

┌──(root㉿kali)-[~]
└─#
```

Для того, чтобы проверить сетевые подключения, введем nmcli con show.

```
┌──(root㉿kali)-[~]
└─# nmcli con show
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  f18d30c9-a5ce-32b9-9c9a-649147e70acd  ethernet  eth0
lo                  51dab5f5-325a-462b-af9d-e26fe1576013  loopback  lo
VPN connection 1    9645089b-c358-4950-877a-71e8e5869ed8  vpn       --

┌──(root㉿kali)-[~]
```

Если использовать опцию -a, то будут видны только активные соединения.

```
┌──(root㉿kali)-[~]
└─# nmcli con show -a
NAME                UUID                                  TYPE      DEVICE
Wired connection 1  f18d30c9-a5ce-32b9-9c9a-649147e70acd  ethernet  eth0
lo                  51dab5f5-325a-462b-af9d-e26fe1576013  loopback  lo

┌──(root㉿kali)-[~]
```

1.7 ethtool

Для начала узнаем какие у нас есть сетевые карты:

```
┌──(root㉿kali)-[~]
└─# lspci | grep Eth
00:03.0 Ethernet controller: Intel Corporation 82540EM Gigabit Ethernet Controller (rev 02)

┌──(root㉿kali)-[~]
└─#
```

При помощи ethtool мы можем посмотреть информацию о сетевой карте, используя команду ethtool (# ens33). Будет примерно такой вывод:

```
root@debian:/# ethtool ens37
Settings for ens37:
        Supported ports: [ TP ]
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Supported pause frame use: No
        Supports auto-negotiation: Yes
        Supported FEC modes: Not reported
        Advertised link modes:  10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Advertised pause frame use: No
        Advertised auto-negotiation: Yes
        Advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Port: Twisted Pair
        PHYAD: 0
        Transceiver: internal
        Auto-negotiation: on
        MDI-X: off (auto)
        Supports Wake-on: d
        Wake-on: d
        Current message level: 0x00000007 (7)
                               drv probe link
        Link detected: yes
```

При необходимости можно сменить рабочую скорость карты.

```
root@debian:/# ethtool -s ens37 speed 100 duplex full
root@debian:/#
root@debian:/# ethtool ens37
Settings for ens37:
        Supported ports: [ TP ]
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
                                1000baseT/Full
        Supported pause frame use: No
        Supports auto-negotiation: Yes
        Supported FEC modes: Not reported
        Advertised link modes:  100baseT/Full
        Advertised pause frame use: No
        Advertised auto-negotiation: Yes
        Advertised FEC modes: Not reported
        Speed: 1000Mb/s
        Duplex: Full
        Port: Twisted Pair
        PHYAD: 0
        Transceiver: internal
        Auto-negotiation: on
        MDI-X: off (auto)
        Supports Wake-on: d
        Wake-on: d
        Current message level: 0x00000007 (7)
                               drv probe link
        Link detected: yes
```

2. Инструменты сетевого сканирования и анализа производительности

2.1 netstat

netstat - это инструмент для отображения информации, такой как сетевые соединения, таблицы маршрутизации, статистика интерфейса и тп. Это основной инструмент отладки сетевых служб, используемый для проверки того, какие программы какие порты прослушивают.

Например можно посмотреть все порты TCP в режиме прослушивания и программы, которые их прослушивают.

```
┌──(root㉿kali)-[~]
└─# netstat -tnlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      770/sshd: /usr/sbin
tcp6       0      0 :::22                   :::*                    LISTEN      770/sshd: /usr/sbin

┌──(root㉿kali)-[~]
└─#
```

Чтобы посмотреть таблицу маршрутизации ядра используем опцию -r.

```
┌──(root㉿kali)-[~]
└─# netstat -r
Kernel IP routing table
Destination     Gateway         Genmask         Flags   MSS Window  irtt Iface
default         10.0.2.0        0.0.0.0         UG        0 0          0 eth0
default         10.0.2.2        0.0.0.0         UG        0 0          0 eth0
10.0.2.0        0.0.0.0         255.255.255.0   U         0 0          0 eth0
10.10.2.20      -               255.255.255.255 !H        - -          - -

┌──(root㉿kali)-[~]
└─# ss
```

Технически netstat устарел, его замена - ss.

2.2 ss

ss (статистика сокетов) - утилита для исследования сокетов. Она сбрасывает статистику сокетов и отображает информацию, аналогичную netstat.

Выведем список TCP-портов (сокетов), открытых сейчас:

```
┌──(root㉿kali)-[~]
└─# ss -ta
State       Recv-Q      Send-Q            Local Address:Port            Peer Address:Port
LISTEN      0           128                  0.0.0.0:ssh                  0.0.0.0:*
LISTEN      0           128                     [::]:ssh                     [::]:*

┌──(root㉿kali)-[~]
```

Посмотрим все активные TCP-соединения вместе с их таймерами.

```
┌──(root㉿kali)-[~]
└─# ss -to
State       Recv-Q      Send-Q            Local Address:Port            Peer Address:Port

┌──(root㉿kali)-[~]
```

Опции ss:
Для сетевых подключений в Linux с помощью утилиты ss можно использовать такие опции:

- **V** — Version показать версию утилиты
- **n** — Numeric не определять имена служб
- **r** — Resolve определять сетевые имена адресов с помощью dns
- **a** — All отобразить все сокеты (открытые соединения)
- **l** — Listening показать только прослушиваемые сокеты
- **o** — Options показать информацию таймера
- **e** — Extended выводить расширенную информацию о сокете
- **p** — Processes, показать процессы, использующие сокет
- **i —** Internal, посмотреть внутреннюю информацию TCP
- **s** — Summary, статистика использования сокета
- **D** — экспортировать текущее состояние TCP сокетов в файл
- **F** — работать с информацией, взятой из файла
  Кроме того, можно вывести сокеты только нужного протокола:

- **4, —ipv4** — только сокеты протокола IP версии 4
- **6 —ipv6** — только сокеты протокола IP версии 6
- **0, —packet** — только PACKET сокеты
- **t, —tcp** — TCP сокеты
- **u, —udp** — UDP сокеты
- **d, —dhcp** — DHCP сокеты
- **r, —raw** — RAW сокеты
- **x, —unix** — UNIX сокеты

2.3 NC (NetCat)

NC (NetCat) - "сетевой швейцарский армейский нож" - утилита для решения задач с сокетами TCP, UDP или UNIX-домена. Он используется для открытия TCP-соединений, прослушивания произвольных TCP- и UDP-портов, выполнения сканирования и тп. Также может быть использован в качестве простого TCP-прокси, для тестирования сетевых демонов, проверки доступности удаленных портов и тп.
Также можно использовать netcat совместно с pv для передачи файлов между 2-мя компьютерами.

Попробуем просканировать список портов (примерно так все должно выглядеть):

```
igor@debian:~$ nc -zv 192.168.127.167 21 22 80 443 3000
192.168.127.167: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.127.167] 22 (ssh) open
```

```
┌──(root㉿kali)-[~]
└─# nc -zv 192.168.127.167 21 22 80 443 3000
192.168.127.167: inverse host lookup failed: Unknown host
```

Также можно указать диапазон портов:

```
igor@debian:~$ nc -zv 192.168.127.167 20-90
192.168.127.167: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.127.167] 53 (domain) open
(UNKNOWN) [192.168.127.167] 22 (ssh) open
```

```
┌──(kali㉿kali)-[~]
└─$ nc -zv server2.tecmint.lan 20-240
server2.tecmint.lan: forward host lookup failed: Unknown host
```

Можно использовать nc для открытия TCP-соединения с портом 22 на 192.168.127.167, используя при этом порт 3000 в качестве исходного порта с тайм-аутом в 10 секунд.

```
nc -p 3000 -w 10 192.168.127.167 22
```

Общий вид команды netcat:
nc (параметры) (ip-адрес) (порты)

Параметры:

- 6 – использовать протокол IPv6. По умолчанию используется параметр -4 и IPv4 соответственно;
- h – вывести справку со списком доступных параметров;
- i (задержка) – добавить задержку между отправкой строк или сканированием портов. Задаётся в секундах;
- l – режим прослушивания. Используется с указанием порта;
- N – закрыть соединение при достижении конца файла при его отправке;
- n – Работать с IP-адресами напрямую, не задействуя DNS, также отключить поиск портов;
- P имя_пользователя – указать имя пользователя для подключения к прокси;
- x (адрес : порт) – указать адрес и порт для подключения к прокси;
- p порт – указать номер порта. В большинстве случаев порт считывается без указания параметра;
- U – использовать сокет домена UNIX (для межпроцессного взаимодействия);
- u – использовать протокол UDP, по умолчанию используется TCP;
- v – подробный режим. Используется при сканировании портов;
- W (количество пакетов) – закрыть соединение после получения определённого количества пакетов;
- w (таймер) – включить таймер для ограничения времени соединения. Задаётся в секундах;
- z – отключить отправку данных. Используется при сканировании портов.

2.4 Nmap

Nmap (Network Mapper) - инструмент для сбор информации об одном узле или для исследования сетей в целом. Также он используется для сканирования безопасности, аудита сети, поиска открытых портов на удаленных хостах и тп.

Просканируем хост, используя его имя:

```
┌──(root㉿kali)-[~]
└─# nmap google.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-17 13:35 EDT
Nmap scan report for google.com (209.85.233.101)
Host is up (0.013s latency).
Other addresses for google.com (not scanned): 209.85.233.138 209.85.233.102 209.85.233.139 209.85.233.100 209.85
.233.113 2a00:1450:4010:c03::8b 2a00:1450:4010:c03::66 2a00:1450:4010:c03::64 2a00:1450:4010:c03::71
rDNS record for 209.85.233.101: lr-in-f101.1e100.net
Not shown: 998 filtered tcp ports (no-response)
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https

Nmap done: 1 IP address (1 host up) scanned in 5.43 seconds

┌──(root㉿kali)-[~]
```

Синтаксис:
nmap (опции) (адрес)

Основные опции:

- **sL** - просто создать список работающих хостов, но не сканировать порты nmap;
- **sP** - только проверять доступен ли хост с помощью ping;
- **PN** - считать все хосты доступными, даже если они не отвечают на ping;
- **sS/sT/sA/sW/sM** - TCP сканирование;
- **sU** - UDP сканирование nmap;
- **sN/sF/sX** - TCP NULL и FIN сканирование;
- **sC** - запускать скрипт по умолчанию;
- **sI** - ленивое Indle сканирование;
- **-p** - указать диапазон портов для проверки;
- **sV** - детальное исследование портов для определения версий служб;
- **O** - определять операционную систему;
- **T[0-5]** - скорость сканирования, чем больше, тем быстрее;
- **D** - маскировать сканирование с помощью фиктивных IP;
- **S** - изменить свой IP адрес на указанный;
- **e** - использовать определенный интерфейс;
- **-spoof-mac** - установить свой MAC адрес;
- **A** - определение операционной системы с помощью скриптов.

Простое выполнение трассировки до хоста:

```
┌──(root㉿kali)-[~]
└─# nmap --traceroute google.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-17 13:37 EDT
Nmap scan report for google.com (209.85.233.102)
Host is up (0.0049s latency).
Other addresses for google.com (not scanned): 209.85.233.100 209.85.233.113 209.85.233.138 209.85.233.101 209.85
.233.139 2a00:1450:4010:c03::71 2a00:1450:4010:c03::65 2a00:1450:4010:c03::66 2a00:1450:4010:c03::8a
rDNS record for 209.85.233.102: lr-in-f102.1e100.net
Not shown: 998 filtered tcp ports (no-response)
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https

TRACEROUTE (using port 80/tcp)
HOP RTT     ADDRESS
1   2.27 ms 10.0.2.2
2   2.30 ms lr-in-f102.1e100.net (209.85.233.102)

Nmap done: 1 IP address (1 host up) scanned in 5.64 seconds

┌──(root㉿kali)-[~]
```

Трассировка без сканирования:

```
┌──(root㉿kali)-[~]
└─# nmap --traceroute -sn  google.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-17 13:38 EDT
Nmap scan report for google.com (209.85.233.113)
Host is up (0.00024s latency).
Other addresses for google.com (not scanned): 209.85.233.138 209.85.233.101 209.85.233.139 209.85.233.102 209.85
.233.100 2a00:1450:4010:c03::66 2a00:1450:4010:c03::8a 2a00:1450:4010:c03::71 2a00:1450:4010:c03::65
rDNS record for 209.85.233.113: lr-in-f113.1e100.net

TRACEROUTE (using port 80/tcp)
HOP RTT     ADDRESS
1   0.14 ms 10.0.2.2
2   0.21 ms lr-in-f113.1e100.net (209.85.233.113)

Nmap done: 1 IP address (1 host up) scanned in 0.16 seconds

┌──(root㉿kali)-[~]
```

Вывод полных данных при трассировке:

```
┌──(root㉿kali)-[~]
└─# nmap --traceroute -sn -PE  google.com
Starting Nmap 7.95 ( https://nmap.org ) at 2025-07-17 13:38 EDT
Nmap scan report for google.com (209.85.233.101)
Host is up (0.062s latency).
Other addresses for google.com (not scanned): 209.85.233.139 209.85.233.102 209.85.233.100 209.85.233.113 209.85
.233.138 2a00:1450:4010:c03::66 2a00:1450:4010:c03::8a 2a00:1450:4010:c03::71 2a00:1450:4010:c03::65
rDNS record for 209.85.233.101: lr-in-f101.1e100.net

TRACEROUTE (using proto 1/icmp)
HOP RTT      ADDRESS
1   0.22 ms  10.0.2.2
2   1.64 ms  192.168.0.1
3   4.12 ms  100.64.0.1
4   3.80 ms  91.211.236.126
5   4.93 ms  91.211.236.121
6   4.45 ms  91.211.236.74
7   4.60 ms  178.178.107.1
8   52.63 ms 10.222.70.73
9   52.12 ms 10.222.70.1
10  53.48 ms 216.239.49.39
11  52.57 ms 192.178.241.66
12  66.14 ms 192.178.240.239
13  63.77 ms 172.253.66.110
14  63.56 ms 142.250.56.219
15  ... 23
24  62.10 ms lr-in-f101.1e100.net (209.85.233.101)

Nmap done: 1 IP address (1 host up) scanned in 3.54 seconds

┌──(root㉿kali)-[~]
```

2.5 host

host - команда для поиска DNS, которая переводит имена хостов в IP-адреса и наоборот.

```
┌──(root㉿kali)-[~]
└─# host ya.ru
ya.ru has address 77.88.44.242
ya.ru has address 77.88.55.242
ya.ru has address 5.255.255.242
ya.ru has IPv6 address 2a02:6b8::2:242
ya.ru mail is handled by 10 mx.yandex.ru.

┌──(root㉿kali)-[~]
```

2.6 dig

dig (поиск информации о домене) - утилита поиска DNS, которая используется для запроса информации, связанной с DNS, такой как A Record, CNAME, MX Record и тп.

Примеры применения:

dig ya.ru

```
┌──(root㉿kali)-[~]
└─# dig ya.ru

; <<>> DiG 9.20.9-1-Debian <<>> ya.ru
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 203
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;ya.ru.                         IN      A

;; ANSWER SECTION:
ya.ru.                  532     IN      A       77.88.44.242
ya.ru.                  532     IN      A       77.88.55.242
ya.ru.                  532     IN      A       5.255.255.242

;; Query time: 3 msec
;; SERVER: 192.168.0.1#53(192.168.0.1) (UDP)
;; WHEN: Thu Jul 17 13:42:01 EDT 2025
;; MSG SIZE  rcvd: 82


┌──(root㉿kali)-[~]
```

dig google.com

```
┌──(root㉿kali)-[~]
└─# dig google.com

; <<>> DiG 9.20.9-1-Debian <<>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 32534
;; flags: qr rd ra; QUERY: 1, ANSWER: 6, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;google.com.                    IN      A

;; ANSWER SECTION:
google.com.             190     IN      A       209.85.233.102
google.com.             190     IN      A       209.85.233.138
google.com.             190     IN      A       209.85.233.101
google.com.             190     IN      A       209.85.233.100
google.com.             190     IN      A       209.85.233.113
google.com.             190     IN      A       209.85.233.139

;; Query time: 11 msec
;; SERVER: 192.168.0.1#53(192.168.0.1) (UDP)
;; WHEN: Thu Jul 17 13:42:36 EDT 2025
;; MSG SIZE  rcvd: 135


┌──(root㉿kali)-[~]
```

Синтаксис:

dig (сервер) (доменное имя) (тип записи) (флаги)

где:

- сервер — IP-адрес или доменное имя DNS-сервера (если не указано, dig будет обращаться к DNS-серверу, используемому по умолчанию);
- доменное имя — доменное имя интернет-ресурса, о котором необходимо получить информацию;
- тип записи— позволяет указать, для какого типа записи необходим вывод, например **A**, **NS**, **MX** или **TXT**;
- флаги — с помощью флагов утилите dig отдаются дополнительные команды; оговаривается, каким должен быть вывод команды.

Опции:

- **+[no]all** — отображает или скрывает все установленные по умолчанию флаги отображения;
- **+[no]answer** — отображает только ответ на запрос;
- **+[no]fail** — эта опция указывает, должна ли утилита переключаться на следующий DNS сервер, если текущий не отвечает (по умолчанию стоит **+fail);**
- **+short** — сокращает вывод утилиты;
- **+[no]cmd** — отключает вывод заголовка и информации об использованных опциях утилиты;
- **+[no]identify** — используется вместе с флагом **+short** и отображает информацию об IP-адресе сервера;
- **+[no]comments** — удаляет все комментарии из вывода утилиты;
- **+[no]trace** - позволяет вывести список DNS серверов через которые прошёл запрос на получение информации о домене, по умолчанию отключено.

А также (можно применять вместе с dig):

- **4** — позволяет использовать только IPv4;
- **6** — позволяет использовать только IPv6;
- **x** — предназначена для получения домена по IP;
- **f** — используется для чтения списка доменов из файла;
- **b IP-адрес** — позволяет указать исходящий IP-адрес, с которого отправлен запрос к DNS-серверу, полезно, если к компьютеру подключено несколько сетевых карт;
- **r** — предотвращает чтение настроек из файла ~/.digrc;
- **t** — позволяет указать тип записи, которую надо получить;
- **p** — позволяет указать номер порта DNS сервера;
- **u** — отображает время в секундах вместо миллисекунд.

2.7 Nslookup

Nslookup - утилита для интерактивного и неинтерактивного запроса DNS-серверов. Она используется для запроса записей ресурсов DNS (PR).

```
┌──(root㉿kali)-[~]
└─# nslookup ya.ru
Server:         192.168.0.1
Address:        192.168.0.1#53

Non-authoritative answer:
Name:   ya.ru
Address: 77.88.44.242
Name:   ya.ru
Address: 77.88.55.242
Name:   ya.ru
Address: 5.255.255.242
Name:   ya.ru
Address: 2a02:6b8::2:242


┌──(root㉿kali)-[~]
```

```
┌──(root㉿kali)-[~]
└─# nslookup googlew.com
Server:         192.168.0.1
Address:        192.168.0.1#53

Non-authoritative answer:
Name:   googlew.com
Address: 107.161.23.204
Name:   googlew.com
Address: 198.251.81.30
Name:   googlew.com
Address: 209.141.38.71


┌──(root㉿kali)-[~]
```

Синтаксис:
nslookup (опции) (домен) (сервер)

В таком контексте домен - это то доменное имя, для которого необходимо посмотреть информацию, а сервер - это необязательный параметр, который указывает, что нужно использовать другой DNS сервер.

Опции:

- **type** - тип информации, которую хотим получить, возможные типы: txt, soa, ptr, ns, mx, mr, minfo, mg, mb, hinfo, gid, cname, a, any;
- **port** - другой порт DNS сервера;
- **recurse** - использоваться другие DNS серверы, если на этом нет ответа;
- **retry** - количество попыток получить нужную информацию;
- **timeout** - время между попытками запросов к серверу;
- **fail** - пробовать другой сервер имен, если этот вернул ошибку.

3. Анализаторы сетевых пакетов

3.1 Tcpdump

Tcpdump - широко используемый сетевой сниффер. Он используется для захвата и анализа пакетов TCP/IP, переданных или полученных по сети на определенном интерфейсе.

Чтобы перехватить пакет надо его указать при помощи параметра -i.

```
igor@debian:~$ sudo tcpdump -i ens33
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
12:26:05.338724 IP lo-in-f102.1e100.net.https > 192.168.127.167.50982: Flags [R.], seq 1948984350, ack 1282223573, win 64240, length 0
12:26:05.340059 IP 192.168.127.167.33626 > 104.16.249.249.https: Flags [P.], seq 1692419634:1692419690, ack 191602185, win 62780, length 56
12:26:05.340118 IP 192.168.127.167.33626 > 104.16.249.249.https: Flags [P.], seq 56:139, ack 1, win 62780, length 83
12:26:05.340118 IP 104.16.249.249.https > 192.168.127.167.33626: Flags [.], ack 56, win 64240, length 0
12:26:05.340129 IP 104.16.249.249.https > 192.168.127.167.33626: Flags [.], ack 139, win 64240, length 0
12:26:05.393986 IP debian.44242 > _gateway.domain: 14517+ PTR? 167.127.168.192.in-addr.arpa. (46)
12:26:05.407820 IP _gateway.domain > debian.44242: 14517 NXDomain 0/0/0 (46)
12:26:05.408356 IP debian.53778 > _gateway.domain: 2315+ PTR? 102.222.194.173.in-addr.arpa. (46)
12:26:05.425463 IP 104.16.249.249.https > 192.168.127.167.33626: Flags [P.], seq 1:36, ack 139, win 64240, length 35
12:26:05.425703 IP 192.168.127.167.33626 > 104.16.249.249.https: Flags [.], ack 36, win 62780, length 0
12:26:05.464958 IP _gateway.domain > debian.53778: 2315 1/4/8 PTR lo-in-f102.1e100.net. (338)
12:26:05.465264 IP debian.51940 > _gateway.domain: 53002+ PTR? 249.249.16.104.in-addr.arpa. (45)
12:26:05.479183 IP 104.16.249.249.https > 192.168.127.167.33626: Flags [P.], seq 36:265, ack 139, win 64240, length 229
12:26:05.479316 IP 192.168.127.167.33626 > 104.16.249.249.https: Flags [.], ack 265, win 62780, length 0
12:26:05.480154 IP 192.168.127.167.50990 > lo-in-f102.1e100.net.https: Flags [S], seq 4279010022, win 64240, options [mss 1460,sackOK,TS val 4166086620 ecr 0,n
p,wscale 7], length 0
12:26:05.523692 IP _gateway.domain > debian.51940: 53002 NXDomain 0/1/0 (107)
12:26:05.524356 IP debian.46286 > _gateway.domain: 32118+ PTR? 2.127.168.192.in-addr.arpa. (44)
12:26:05.533663 IP lo-in-f102.1e100.net.https > 192.168.127.167.50990: Flags [S.], seq 1090985772, ack 4279010023, win 64240, options [mss 1460], length 0
12:26:05.533865 IP 192.168.127.167.50990 > lo-in-f102.1e100.net.https: Flags [.], ack 1, win 64240, length 0
12:26:05.535568 IP 192.168.127.167.50990 > lo-in-f102.1e100.net.https: Flags [P.], seq 1:518, ack 1, win 64240, length 517
12:26:05.535708 IP lo-in-f102.1e100.net.https > 192.168.127.167.50990: Flags [.], ack 518, win 64240, length 0
12:26:05.585297 IP _gateway.domain > debian.46286: 32118 NXDomain 0/1/0 (99)
12:26:05.585512 IP debian.34357 > _gateway.domain: 5187+ PTR? 151.127.168.192.in-addr.arpa. (46)
12:26:05.590529 IP _gateway.domain > debian.34357: 5187 NXDomain 0/0/0 (46)
12:26:06.231919 ARP, Request who-has 192.168.127.158 tell debian, length 28
12:26:06.332910 IP debian.33512 > _gateway.domain: 50803+ PTR? 158.127.168.192.in-addr.arpa. (46)
12:26:06.405944 IP debian.47188 > 104.16.249.249.https: Flags [P.], seq 1026703777:1026703816, ack 1553817641, win 62780, length 39
12:26:06.406198 IP 104.16.249.249.https > debian.47188: Flags [.], ack 39, win 64240, length 0
12:26:06.430617 IP _gateway.domain > debian.33512: 50803 NXDomain 0/1/0 (101)
12:26:06.488118 IP 104.16.249.249.https > debian.47188: Flags [P.], seq 1:40, ack 39, win 64240, length 39
12:26:06.488138 IP debian.47188 > 104.16.249.249.https: Flags [.], ack 40, win 62780, length 0
^C
31 packets captured
31 packets received by filter
0 packets dropped by kernel
```

Также можно захватывать и сохранять пакеты в файл для последующего анализа, используя флаг -w.

```
┌──(root㉿kali)-[~]
└─# tcpdump -w tcp.txt -i eth0
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 byt
^C25 packets captured
25 packets received by filter
0 packets dropped by kernel

┌──(root㉿kali)-[~]
└─# ls
Desktop  Documents  Downloads  impacket  mitm6  Music  Pictures  Public  tcp.txt  T
```

Синтаксис:
tspdump (опции) -i (интерфейс) (фильтры)

При вызове необходимо передавать интерфейс, который мы будет отслеживать Если не указать интерфейс, то будет указан первый в списке.
При помощи опций можно настроить отображение и основные возможности утилиты, а при помощи фильтров можно отсеивать ненужные пакеты.

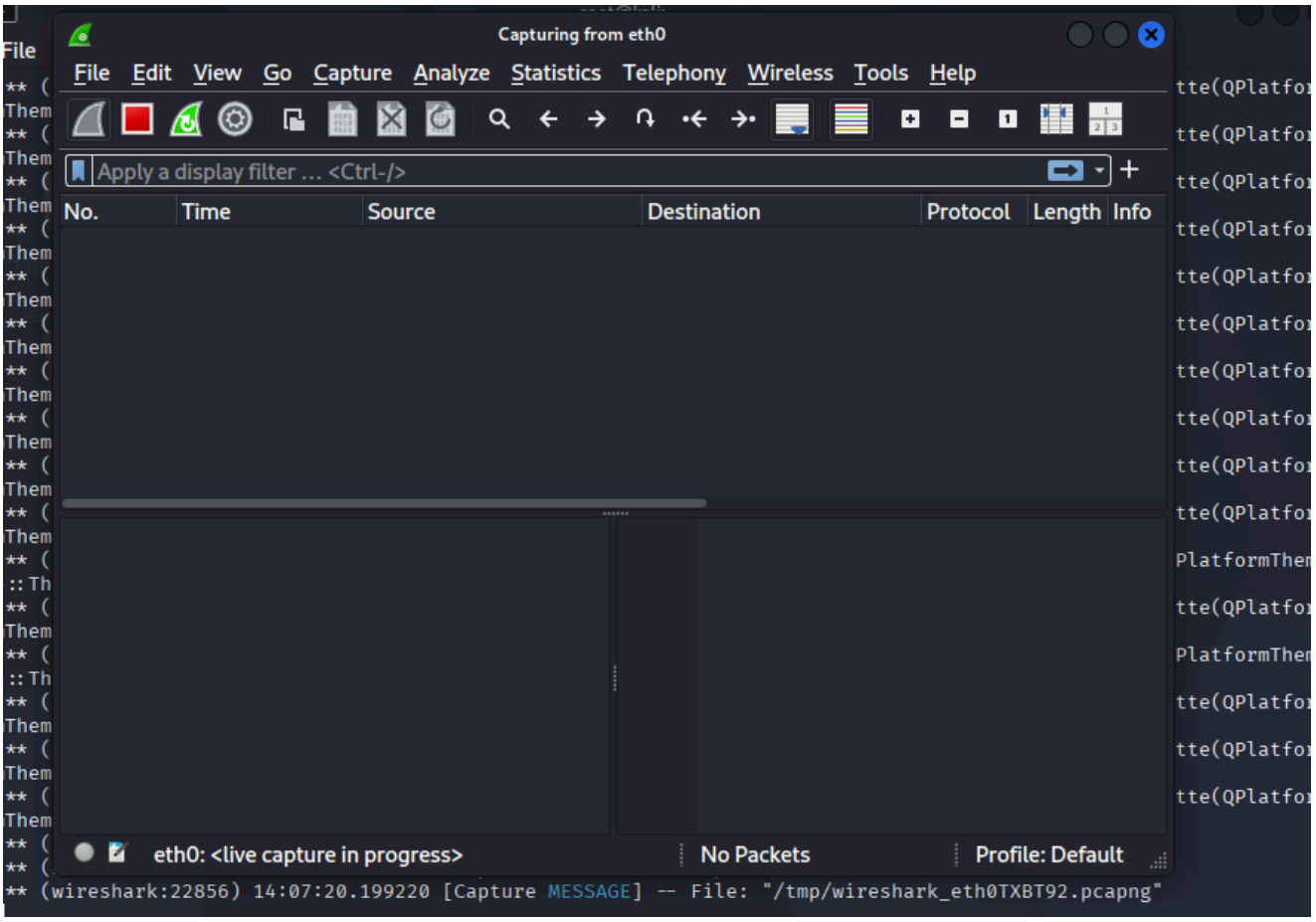
Основные опции:

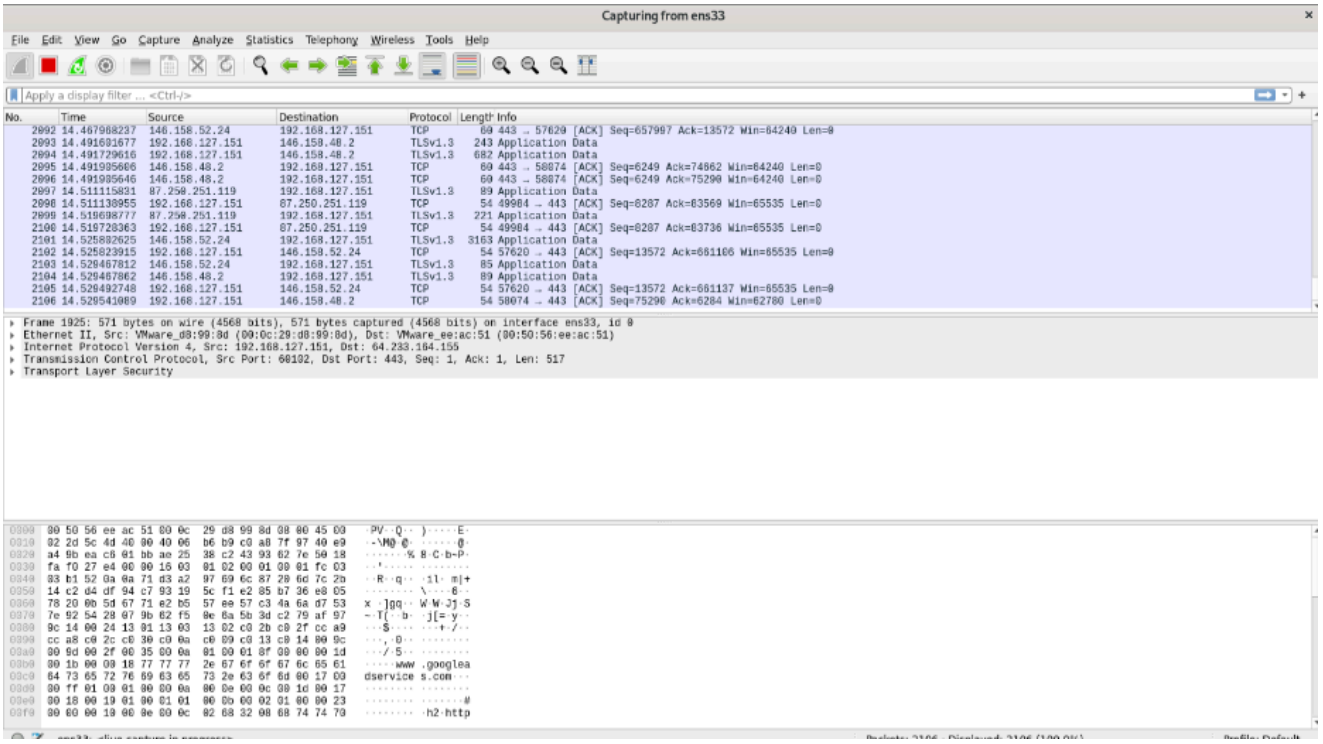
- **A** - выводить все пакеты в формате ASCII;
- **c** - закрыть программу после перехвата n-ого количества пакетов;
- **C** - при записи пакетов в файл, проверять размер файла, и если он больше заданного - создать новый файл;
- **D** - вывести список доступных сетевых интерфейсов;

- **e** - выводить информацию уровня соединения для каждого пакета, это может быть полезно, например, для отображения MAC адреса;
- **f** - выводить доменное имя для ip адресов;
- **F** - читать пакеты из файла, а не интерфейса;
- **G** - создавать новый файл лога через указанный промежуток времени;
- **H** - обнаруживать заголовки 802.11s;
- **i** - имя интерфейса для перехвата пакетов. Вы можете захватывать пакеты со всех интерфейсов, для этого укажите any;
- **I** - переключить интерфейс в режим монитора для захвата всех проходящих пакетов;
- **j** - установить формат Timestamp для записи пакетов;
- **J** - посмотреть доступные Timestamp;
- **K** - не проверять контрольные суммы пакетов;
- **l** - добавить поддержку прокрутки к выводу;
- **L** - вывести поддерживаемые протоколы подключения для интерфейса;
- **n** - не отображать доменные имена;
- **r** - прочитать пакеты из файла, созданного с помощью -w;
- **v, -vv, -vvv** - более подробный вывод;
- **q** - выводить минимум информации;
- **w** - записать вывод в файл;
- **Z** - пользователь, от имени которого будут создаваться файлы.

3.2 Wireshark

Wireshark - универсальный и простой в использовании инструмент захвата и анализа пакетов в вести с коммутацией пакетов в режиме реального времени.





Можно сохранять полученные файлы для последующего просмотра. Wireshark используется для мониторинга и проверки пакетов в целях безопасности и устранения неполадок.

3.3 bmon

bmon - это утилита для сетевого мониторинга и отладки. Она собирает статистику, связанную с сетью, а также выводит ее визуально в удобном для человека формате.

Это эффективный монитор полосы пропускания и оценщик скорости в реальном времени.

4. Инструменты управления брандмауэром Linux

Iptables - это инструмент для настройки, обслуживания и проверки таблиц фильтрации IP-пакетов, а также набора правил NAT. Он используется для настройки и управления брандмауэром Linux (NetFilter). Благодаря этому можно перечислить существующие правила фильтрации пакетов, добавлять, удалять и изменять правила фильтрации

пакетов, список счетчиков правил фильтрации пакетов.

```
┌──(root㉿kali)-[~]
└─# iptables -h
iptables v1.8.11 (nf_tables)

Usage: iptables -[ACD] chain rule-specification [options]
       iptables -I chain [rulenum] rule-specification [options]
       iptables -R chain rulenum rule-specification [options]
       iptables -D chain rulenum [options]
       iptables -[LS] [chain [rulenum]] [options]
       iptables -[FZ] [chain] [options]
       iptables -[NX] chain
       iptables -E old-chain-name new-chain-name
       iptables -P chain target [options]
       iptables -h (print this help information)

Commands:
Either long or short options are allowed.
  --append  -A chain            Append to chain
  --check   -C chain            Check for the existence of a rule
  --delete  -D chain            Delete matching rule from chain
  --delete  -D chain rulenum
                                Delete rule rulenum (1 = first) from chain
  --insert  -I chain [rulenum]
                                Insert in chain as rulenum (default 1=first
  --replace -R chain rulenum
                                Replace rule rulenum (1 = first) in chain
  --list    -L [chain [rulenum]]
                                List the rules in a chain or all chains
  --list-rules -S [chain [rulenum]]
                                Print the rules in a chain or all chains
  --flush   -F [chain]          Delete all rules in  chain or all chains
  --zero    -Z [chain [rulenum]]
                                Zero counters in chain or all chains
  --new     -N chain            Create a new user-defined chain
```